SIMPLE EXPENSE TRACKER

Project Report · Python, Flask, CSV & Render

Project type	Responsive web application
Primary purpose	Record expenses and summarize spending
Storage format	CSV file using Python's csv module
Deployment target	Render Web Service

Prepared as a project handover and submission report. Add your name, course/internship details, repository URL, and live deployment URL before final submission if requested.

1. Project Overview

The Simple Expense Tracker is a lightweight web application that helps users record everyday expenses and understand spending patterns. A user can enter an expense date, category, description, and amount. The dashboard calculates total spending and category-wise totals, and supports month-based filtering.

2. Objectives

• Apply Python data structures, functions, validation, and file handling to a practical problem.

• Persist expense records in a structured CSV file.

• Generate total and category-wise spending summaries.

• Provide a responsive interface suitable for desktop and mobile screens.

• Package the application for deployment as a Render web service.

3. Technology Stack

• Python 3: application logic, validation, file handling, and calculations.

• Flask: HTTP routes, form handling, page rendering, and flash messages.

• CSV: simple structured local data storage and export.

• HTML and CSS: accessible form controls, dashboard cards, category bars, and expense table.

• Gunicorn: production WSGI server used by the Render start command.

• Render: cloud hosting target configured with render.yaml.

4. Key Features

• Create expense records containing date, category, description, and amount.

• Validate dates, accepted categories, required descriptions, and positive numeric amounts.

• Display overall or selected-month spending and the number of records.

• Calculate category totals and percentage of displayed spending.

• Filter records by month and clear the filter to return to the full history.

• Delete individual expense records after a confirmation prompt.

• Export the stored CSV file and expose a /health endpoint for deployment checks.

5. Application Design

The Flask application is organized around a small set of routes. The home route reads records and prepares summary data. The add route validates submitted fields and appends a record. The delete route removes the selected record. The export route serves the CSV file as a download, while the health route returns a simple status response. Storage helpers centralize reading and writing so calculations are separate from persistence.

6. Data Model

Each CSV record contains five fields: id, date, category, description, and amount. The date is stored in ISO format (YYYY-MM-DD), and the amount is stored as a positive decimal value rounded to two places. A unique numeric ID is used by the delete action. CSV is human-readable and easy to export, making it appropriate for a beginner project.

Field	Purpose
id	Unique record identifier
date	Date of expense (YYYY-MM-DD)
category	Expense category
description	Short label explaining the expense
amount	Positive amount, to two decimal places

7. Validation and Error Handling

• Dates must match the YYYY-MM-DD format.

• Categories must be one of the predefined choices.

• Descriptions are required and limited to 120 characters.

• Amounts must be finite, positive, and within the configured maximum.

• Malformed CSV rows are skipped when loading records instead of breaking the dashboard.

• User-facing success and error messages communicate the outcome of actions.

8. Local Setup and Run

On Windows, open PowerShell in the project folder. Create and activate a virtual environment with `py -m venv .venv` and ` .\\.venv\\Scripts\\Activate.ps1 `, install dependencies with `pip install -r requirements.txt`, and start the app with `python app.py`. Browse to http://127.0.0.1:5000. The app creates data/expenses.csv on first use.

9. Deployment Plan

Push the project to GitHub and connect the repository to Render. The included render.yaml defines a Python web service, dependency installation command, Gunicorn start command, and /health health check. For a manual Render service, use `pip install -r requirements.txt` as the build command and `gunicorn app:app` as the start command.

10. Data Persistence Note

The application persists records to a CSV file on the server. However, a file on Render's ephemeral filesystem is not guaranteed to survive redeployments or restarts. For durable hosted records, configure a supported persistent disk and set DATA_DIR to its mount path, or move storage to a managed database such as PostgreSQL. Verify the plan's storage features before relying on cloud data retention.

11. Testing Checklist

• Add a valid expense and verify that it appears in the expense history.

• Try blank descriptions, invalid dates, and zero or negative amounts.

• Check that totals and category percentages update after adding or deleting records.

• Apply a month filter and confirm only matching dates appear.

• Export CSV and verify that the downloaded file contains the saved fields.

• Open /health and confirm it returns a JSON status of ok.

• After deployment, repeat core tests on the live Render URL.

12. Limitations and Future Enhancements

• CSV is not ideal for simultaneous writes or large volumes of data; SQLite or PostgreSQL would be more robust.

• The current app does not include user accounts or authentication.

• Future enhancements could include budget limits, interactive charts, recurring expenses, search, and date-range filtering.

• A production version should use a strong secret key, durable storage, and an explicit backup strategy.

13. Interview Questions and Answers

• How would you store expense records? In this project, each record is a CSV row with an ID, date, category, description, and amount. A relational database is a good next step as complexity grows.

• Why persist application data? Persistence makes records available after a request ends. Cloud deployments also need durable storage that survives instance replacement and redeployments.

• How would monthly filtering be added? Validate a YYYY-MM month value, match it against the start of each ISO date, and calculate summaries from only the matching records.

14. Submission Details

Before submitting, add the final GitHub repository URL and live Render URL to the README or your submission form. Confirm that the repository is public if the internship portal requires public access, include the report if requested, and attach rollback/deployment evidence if required by the assignment guidelines.

End of report